

Digital Signal Processing Lab 1

DSP Basics: Sampling, Convolution and DFT

Nikos Deligiannis,
David Blinder,
Brent De Weerd (brent.de.weerd@vub.be),
and Giannis Dilmperis (ioannis.dilmperis@vub.be)

1 Introduction to Matlab

We usually represent a discrete signal by a vector, where each element (sample) corresponds to the value of the signal at a given time step. For instance, it can be a vector of voltage measurements acquired at regular time intervals. Images can also be considered as digital signals; in this case, we speak about signals varying in 2-D space. As a result, images are represented using matrices instead of vectors, and each matrix element represents the color of the pixel at that position (in the case of RGB images, each matrix element is a 3-dimensional vector itself, representing the red, green and blue intensities of the pixel.)

In this first section, you will get acquainted with basic Matlab operations to generate, manipulate and display such kind of signals.

Exercise 1. Vectors in Matlab

- i. Create the following row vectors:
 $a = [5 \ 7 \ 2 \ 3]$
 $b = [4 \ 6 \ 8 \ 9]$
- ii. Access the first and the last element of each vector typing $a(1)$, $a(\text{end})$, $b(1)$, $b(\text{end})$.
- iii. Calculate the dot product of a and b using the `dot` command.
- iv. Sum the elements of a using `sum` command.
- v. What is the result of $a+b$, $a*5$, $a.*b$?
- vi. Create a row vector with 20 zeros using the `zeros` command.
- vii. Create a column vector with 20 ones using the `ones` command.
- viii. Create the following vector using the `linspace` command
 $t = [1 \ 2 \ 3 \ 4 \ 5]$
- ix. What is the result of the following commands?
 $t = 1:5$
 $t = 1:0.5:5$
 $t = 1:0.3:5$

Exercise 2. Displaying basic signals

- i. Plot one-dimensional signals using the `plot` command.

Create a sinusoid signal running the following commands:

```
f = 100;  
tmin = 0;  
tmax = 0.01;  
t = linspace(tmin, tmax, 100);  
y = sin(2*pi*f*t);
```

Plot the signal you created using the `plot` command:

```
plot(t, y);
```

- ii. Plot one-dimensional signals using the `stem` command.

- (a) Create a unit step signal running the following commands:

```
n = -10:10;  
u = [zeros(10,1); ones(11,1)];
```

- (b) Plot the signal you created:

```
figure;  
stem(n, u, 'filled', 'LineWidth', 1);  
title('Unit step signal');
```

- (c) Create and plot the unit sample sequence.

- (d) Create and plot the unit ramp signal.

- (e) Create a sine signal $y(t)$ with frequency 100Hz defined in $t \in [0, 0.01]$ at 20 points, and plot it using the `stem`. Plot a cosine signal of the same frequency on the same figure.

- iii. Load and display images (two-dimensional signals).

- (a) Load an image using the command `imread`:

```
I = imread('lenagray.png');
```

- (b) Convert the intensity image `I` to double precision:

```
I = im2double(I);
```

- (c) Display image `I` in a figure using the command `imshow`:

```
figure; imshow(I);
```

The image `I` can be grayscale, RGB (truecolor), or binary.

2 Convolution

Exercise 3. The conv command

- i. Create and plot the signal

$$f(n) = u(n) - u(n - 4),$$

where $u(n)$ is the unit step signal.

- ii. Create and plot the signal

$$g(n) = n \cdot u(n) - 2 \cdot (n - 4) \cdot u(n - 4) + (n - 8) \cdot u(n - 8).$$

- iii. Compute the convolution $f(n) * g(n)$ using the matlab command `conv` and plot the result. What is the length of the output signal?

Exercise 4. Implement your own filtering operation

Convolution is the fundamental operation in signal filtering: In figure 1, the original signal is convolved with filter of length 3 (the filter is also called “convolution kernel”). Each output sample $y(n)$ is computed as the sum of the element-wise product of the input signal with the filter, flipped and centered at position n . In discrete signal filtering, we usually want to keep the length of the output equal to the length of the input signal: this is why the original signal in Figure 1 has been padded with a single zero (on both sides).

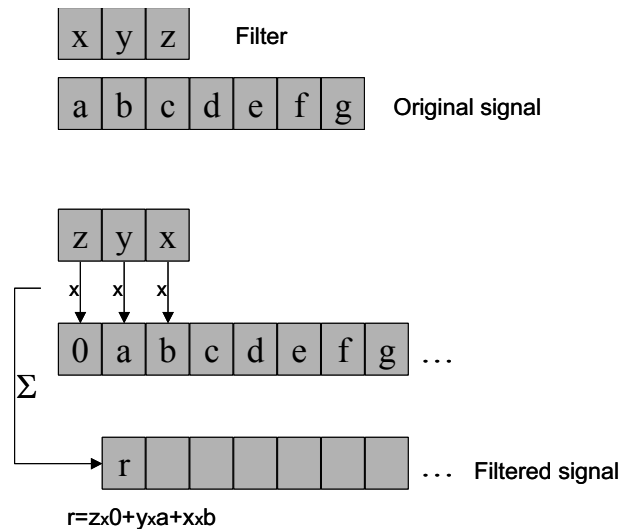


Figure 1: Filtering a one-dimensional signal.

- i. Implement your own filtering operation to perform filtering on a one dimensional signal for filters with an odd number of samples. At this stage, do **not** use the `conv` command. During filtering, your input signal should be first padded with zeros to fill in the borders.
- ii. Specify an average filter with 3 samples ($1/3[1 \ 1 \ 1]$) and filter a random 1-D signal. Compare your results with the matlab `conv` command. (Tip: use the parameter `'same'` of `conv` to compute the central part of the output that is of the same length as the input).

3 Sampling and Quantization

In terms of discretization, digital signals have two fundamental properties:

- The sampling frequency (discretization in time). In this case, the Shannon-Nyquist theorem offers a guarantee for being able to perfectly reconstruct a continuous signal from discrete samples, as long as the sampling frequency is at least twice the maximum frequency contained in the original continuous signal.
- The quantization level (discretization of the signal value). Quantizing the signal values using a low number of bits will unavoidably lead to distortion.

Exercise 5. Sampling, aliasing

Create a cosine signal with frequency 50Hz and plot it. Sample at 800Hz using the commands:

```
T = 1/800; %sampling period
nmin = ceil(tmin/T);
nmax = floor(tmax/T);
n = nmin:nmax;
x1 = cos(2*pi*f*n*T);
```

and plot the sampled signal on the same figure. Repeat the previous commands and sample at 100Hz and 50Hz. What do you notice?

Exercise 6. Uniform Quantization of a 1-D signal

- i. Implement a uniform quantizer. The function should take as input the signal x and the available number of bits N and return the quantized signal:
 - Divide the dynamic range of the signal into L equal intervals of length Q . Note: If you have N bits available to quantize a sample, then there are $L = 2^N$ quantization intervals.
 - Map the i -th interval to the middle value of this interval
 - For each sample $x(n)$ of the input signal, find the index i of the interval to which the sample belongs and set its quantized value using the mapping defined just above.
- ii. Experiment with the sinusoid signals you created in the previous exercises. Try different resolutions ($N = 4, 5, 6, \dots$)
- iii. For every signal calculate the quantization error.

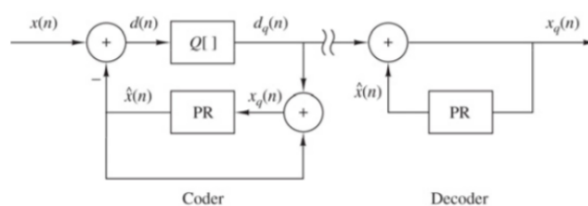
Exercise 7. Uniform Quantization of a 2-D signal

Considering that the pixel values of a grayscale image range from 0 to 255, how many bits do you need for their encoding? Implement a uniform quantizer that encodes the pixel values of a 2-D grayscale image with 4 bits. Load the grayscale image 'lenagray.png', apply 4-bit quantization and display the quantized image.

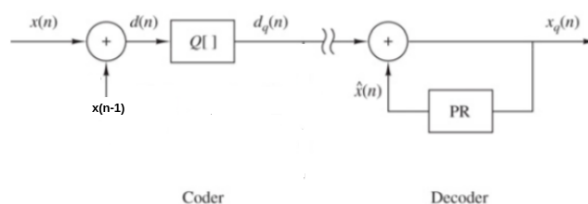
3.1 Exercise 8. DPCM

In this exercise, you will implement a simple DPCM encoding and decoding scheme as studied in Lecture 3 (slide 32). The encoder of the DPCM scheme quantizes the difference between the input signal $x(n)$ and a prediction $\hat{x}(n)$. It is possible to predict $\hat{x}(n)$ in multiple ways. In this case, you will implement two different versions of the encoder:

- The first one (figure 2a) simply uses the reconstructed signal $x_q(n)$ as the prediction for $\hat{x}(n)$
- The second one (figure 2b) does not make use of a prediction scheme. Instead, the error is simply computed as the difference between the sample $x(n)$ with the input sample at the previous time step $x(n-1)$.



(a) DPCM encoder-decoder scheme.



(b) DPCM without prediction in the encoder.

Figure 2

Exercise 8.1 Implement the two encoders mentioned above. For the DPCM encoder, you should follow the skeleton code provided in `'dpcm_encoder.m'`. The code for the decoder is provided in `'dpcm_decoder.m'`.

Exercise 8.2 Build the following input signal:

```
Ts = 1e-3;
N = 300;
t = linspace(0, Ts*N, N)';
f = 10;
x = 0.8 * sin(2*pi*f*3*t) + 0.5 * cos(2*pi*f/5*t)
```

encode it using the DPCM encoder you have implemented in the previous step with and without the prediction. After that, apply the DPCM decoder to get the reconstructions of the signal in the two cases. Plot the two reconstructed signals versus the original one. You should see the *drifting effect* happening in case prediction is not used in the DPCM encoder.

4 Discrete Fourier Transform

Exercise 9: FFT in Matlab

The typical syntax for computing the FFT of a signal is

```
X = fft(x,N)
```

where x is the signal, $x[n]$, you wish to transform, and N is the number of points in the FFT. N must be at least as large as the number of samples in $x[n]$. To demonstrate the effect of changing the value of N , synthesize a cosine with

```
f = 1;  
Fs = 10;  
tmin = 0;  
tmax = 3;
```

using the following commands

```
n1 = tmin/Ts;  
n2 = tmax/Ts;  
n = n1:n2;  
x = cos(2*pi*f*Ts*n);  
figure; plot(n*Ts, x);
```

Define three different values for N . Then take the transform of $x[n]$ for each of the three values that were defined. The `abs` function finds the magnitude of the transform.

```
N1 = 64;  
N2 = 128;  
N3 = 256;  
X1 = abs(fft(x,N1));  
X2 = abs(fft(x,N2));  
X3 = abs(fft(x,N3));
```

The frequency scale begins at 0 and extends to $N - 1$ for an N -point FFT. We then normalize the scale so that it extends from 0 to $1 - \frac{1}{N}$ according to

```
F1 = [0 : N1 - 1]/N1;  
F2 = [0 : N2 - 1]/N2;  
F3 = [0 : N3 - 1]/N3;  
  
figure;  
subplot(3,1,1)  
plot(F1,X1,'-x'),title('N = 64')  
subplot(3,1,2)  
plot(F2,X2,'-x'),title('N = 128')  
subplot(3,1,3)  
plot(F3,X3,'-x'),title('N = 256')
```

Upon examining the plot one can see that each of the transforms adheres to the same shape, differing only in the number of samples used to approximate that shape. What happens if N is the same as the number of samples in $x[n]$?

When the FFT is computed with an N_k larger than the number of samples N in $x[n]$, it fills in the samples after $x[n]$ with zeros. This is like taking a sinusoid and multiplying it with a rectangular box of length N_k . A multiplication of a box and a sinusoid in the time domain should result in the convolution of a sinc with impulses in the frequency domain. Furthermore, increasing the width of the box in the time domain should increase the frequency of the sinc in the frequency domain. The previous Matlab experiment supports this conclusion.

Exercise 10. Spectrum Analysis with the FFT and Matlab (1-D)

The FFT contains information between 0 and F_s , however, we know that the sampling frequency must be at least twice the highest frequency component. Therefore, the signal's spectrum should be entirely below $F_s/2$ the Nyquist frequency. Also a real signal should have a transform magnitude that is symmetrical for positive and negative frequencies. So instead of having a spectrum that goes from 0 to F_s , it would be more appropriate to show the spectrum from $-F_s/2$ to $F_s/2$. This can be accomplished by using Matlab's `fftshift` function.

Apply the `fftshift` function to the FFT transform you computed in the previous section.

```
X = fftshift(X);
F = [-N/2:N/2-1]/N;
figure;
plot(F,X)
```

Exercise 11. Spectrum Analysis with the FFT2 and Matlab (2-D)

Create an image with the use of MATLAB commands that looks like Fig. 3 (128×128). Compute

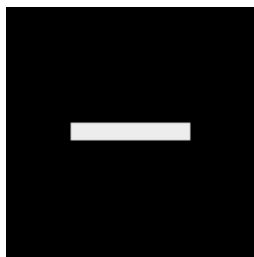


Figure 3: White rectangular.

the spectrum of the image using the command `fft2`. Look at the spectrum of the image with `surf` and `imagesc` (use `fftshift` to get the low frequencies in the middle of the image). What do you notice about the energy present in the highfrequency components of the DFT image? Compare the horizontal and vertical components. What is the reason for the difference? Now rotate the image over 45° , 60° and 90° (`imrotate`). Compare the absolute value of the spectrum of the result with the absolute value of the original spectrum. What do you notice? What property of the DFT is hereby demonstrated?